



Mi Primera APP

Construida desde cero en 90 minutos

FastAPI + React + Vite

Material de trabajo: [GUIA_ALUMNO.md](#) (abierta en tu navegador)

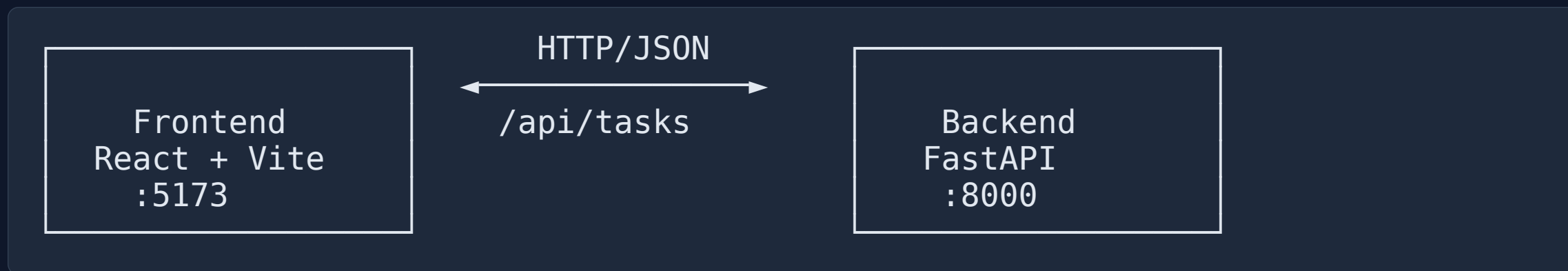
Objetivo del Taller

Construir una app de **Tareas** con API REST y frontend web.

- ▶  **Backend:** API REST con FastAPI en Python — 4 endpoints
- ▶  **Frontend:** Interfaz web con React — consume la API

*El foco NO es aprender frameworks. Es entender el **ciclo HTTP**.*

Arquitectura



Dos procesos independientes que se comunican por **HTTP**.

La API — 4 Operaciones

Método	Ruta	Qué hace
GET	/api/tasks	Listar todas las tareas
POST	/api/tasks	Crear una tarea nueva
PATCH	/api/tasks/{id}	Marcar como hecha / desmarcar
DELETE	/api/tasks/{id}	Eliminar una tarea

Esto es un patrón **CRUD** — Create, Read, Update, Delete.

¿PATCH o PUT?

PATCH — Actualización parcial

Enviás SOLO el campo que cambia.

```
{ "completed": true }
```

PUT — Reemplazo completo

Enviás TODA la tarea.

```
{  
  "id": 1,  
  "title": "Comprar leche",  
  "completed": true,  
  "created_at": "2026-08-04T..."  
}
```

Usamos **PATCH** porque solo cambiamos un campo

Fase 1 — Setup

Verificar herramientas + estructura de directorios

Verificar Herramientas

Cada comando debe responder una versión:

```
python3 --version      # Python 3.12+  
uv --version           # gestor de paquetes Python  
node --version         # Node.js 18+  
pnpm --version         # gestor de paquetes Node
```

¿Falta alguna? Avisá al docente antes de continuar.

Estructura de Directorios

```
mkdir -p 01-mi-primer-app/backend  
mkdir -p 01-mi-primer-app/frontend/src  
mkdir -p 01-mi-primer-app/frontend/public
```

```
01-mi-primer-app/  
├── backend/  
├── frontend/  
│   ├── public/  
│   └── src/
```

Detalles en [GUIA_ALUMNO.md](#) — Parte A

Fase 2 — Backend desde cero

2 archivos, ~215 líneas, 1 servidor corriendo

pyproject.toml — El Manifiesto

Creá `backend/pyproject.toml` — copia de la guía Parte B.1

```
[project]
name = "mi-primer-app-backend"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
    "fastapi>=0.115.0",
    "uvicorn[standard]>=0.30.0",
]
```

¿Qué es? El manifiesto del proyecto Python. `uv` lo lee para instalar las dependencias.

main.py — La API Completa

Creá `backend/main.py` — copió de la guía Parte B.2 (~200 líneas)

Estructura del archivo:

Sección	Qué hace
Modelos Pydantic	<code>TaskCreate</code> (entrada) y <code>Task</code> (salida)
Almacenamiento	Lista en memoria (<code>tasks = []</code>)
Aplicación	Instancia de <code>FastAPI</code> con título y versión
CORS	Permite que el frontend hable con el backend
Endpoints	GET, POST, PATCH, DELETE + health check
Main	Arranca uvicorn con hot-reload

Tip: copió el archivo completo de la guía. No lo escribas de memoria.

Modelos de Datos — Por qué dos

```
class TaskCreate(BaseModel):  
    # Lo que el cliente ENVÍA (solo título)  
    title: str = Field(..., min_length=1, max_length=200)  
  
class Task(BaseModel):  
    # Lo que el servidor DEVUELVE (con ID y metadata)  
    id: int  
    title: str  
    completed: bool  
    created_at: str
```

Separar **entrada** de **salida** es un patrón fundamental. El cliente no necesita enviar el ID — lo genera el servidor.

Instalar y Ejecutar

```
cd 01-mi-primer-app/backend  
uv sync          # instala las dependencias  
uv run main.py   # arranca el servidor
```

Debes ver: `Uvicorn running on http://127.0.0.1:8000`

 *No cierres esta terminal — el servidor queda corriendo.*

Checkpoint 1 — Swagger

Abrí <http://localhost:8000/docs>

Debés ver la interfaz de Swagger con los 4 endpoints.

¿No aparece? Revisá:

- ▶ ¿La terminal muestra errores?
- ▶ ¿Copiaste `main.py` completo?

Esperá al docente antes de avanzar.

Fase 3 — Verificar la API

La API ya está corriendo. Probémosla con 3 clientes distintos.

Probar con Swagger y curl

Swagger (navegador): `http://localhost:8000/docs`

curl (terminal):

```
# Crear una tarea
curl -X POST http://localhost:8000/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title": "Aprender FastAPI"}'

# Listar
curl http://localhost:8000/api/tasks
```

La misma API responde a cualquier cliente. Swagger, curl, Postman — todos hablan HTTP.

Probar con Postman / Bruno

1. Creá una Collection "Mi Primera APP"
2. Agregá los 4 requests

Nombre	Método	URL
Listar	GET	/api/tasks
Crear	POST	/api/tasks
Toggle	PATCH	/api/tasks/1
Eliminar	DELETE	/api/tasks/1

Postman y Bruno son las herramientas que usa la industria para testear APIs.

Fase 4 — Frontend desde cero

7 archivos, ~490 líneas, 1 servidor corriendo

Los 7 Archivos del Frontend

Todos se copian de la guía Parte D, en este orden:

#	Archivo	Qué es
1	<code>package.json</code>	Manifiesto del proyecto (JSON estricto)
2	<code>index.html</code>	El único HTML — React dibuja dentro de <code>#root</code>
3	<code>vite.config.js</code>	Configuración de Vite + proxy a :8000
4	<code>src/main.jsx</code>	Punto de entrada de React
5	<code>src/api.js</code>	Comunicación con el backend (fetch)
6	<code>src/App.jsx</code>	La UI completa
7	<code>src/App.css</code>	Los estilos

¿Por qué a mano y no `pnpm create vite`? Para que entiendas la estructura de verdad.

package.json — JSON es estricto

Creá `frontend/package.json` — copió de la guía Parte D.1

```
{
  "name": "mi-primer-app-frontend",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build"
  },
  "dependencies": {
    "react": "^19.0.0",
    "react-dom": "^19.0.0"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.4.0",
    "vite": "^6.0.0"
  }
}
```

index.html y vite.config.js

Creá `frontend/index.html` y `frontend/vite.config.js` — guía D.2 y D.3

```
<!-- index.html -->
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
```

```
// vite.config.js – el proxy es CLAVE
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      "/api": { target: "http://localhost:8000" }
    }
  }
})
```

El **proxy**: cuando el frontend hace `fetch("/api/tasks")`, Vite lo redirige a `localhost:8000`. Sin esto, el navegador bloquea la petición por CORS.

main.jsx — El Punto de Entrada

Creá `frontend/src/main.jsx` — guía D.4

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App.jsx";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Toma el `<div id="root">` del HTML y monta la app dentro.

api.js — La Capa de Comunicación

Creá `frontend/src/api.js` — guía D.5

```
const API_BASE = "/api";

async function request(url, options = {}) {
  const response = await fetch(`${API_BASE}${url}`, {
    headers: { "Content-Type": "application/json" },
    ...options,
  });
  return response.json();
}

export async function fetchTasks() { return request("/tasks"); }
export async function createTask(title) {
  return request("/tasks", {
    method: "POST",
    body: JSON.stringify({ title }),
  });
}
```

App.jsx — La UI Completa

Creá `frontend/src/App.jsx` — guía D.6 (~160 líneas)

Los 4 conceptos de React que usás:

Concepto	Uso en la app
<code>useState</code>	Guarda la lista de tareas y el input
<code>useEffect</code>	Carga las tareas al montar la app
<code>.map()</code>	Renderiza cada tarea como un elemento
<code>.filter()</code>	Elimina la tarea del estado

Copιά el archivo completo de la guía.

App.css — Los Estilos

Creá `frontend/src/App.css` — guía D.7

```
.app { max-width: 600px; margin: 40px auto; }
.task-item {
  display: flex;
  justify-content: space-between;
  background: white;
  padding: 12px 16px;
  border-radius: 8px;
}
.task-item.completed .task-title {
  text-decoration: line-through;
  color: #999;
}
```

El archivo completo tiene más clases: formulario, botones, errores, estados.

Instalar y Ejecutar

```
cd 01-mi-primer-app/frontend  
pnpm install      # instala React, Vite y el plugin  
pnpm dev          # arranca el dev server
```

Debes ver: → Local: <http://localhost:5173/>

 ***No cierres esta terminal — el frontend queda corriendo.***

Checkpoint 2 — Frontend andando

Abrí `http://localhost:5173`

Debés ver la app con el input para agregar tareas.

¿Aparece la página de Vite pero sin tu app? Revisá:

- ▶ ¿Creaste los archivos en las rutas correctas?
- ▶ ¿La terminal de `pnpm dev` muestra errores?

Esperá al docente antes de avanzar.

Fase 5 — Integración

Los dos servidores trabajando juntos

Verificar la Integración

Con ambos servidores corriendo:

1. Escribí "Comprar leche" y click en **Agregar**
2. La tarea aparece en la lista
3. Marcala como completada — se tacha
4. Eliminala con el botón ×

Bonus: creá una tarea en Swagger y refrescá el frontend — aparece ahí.

¿Funcionó? Construiste una app fullstack desde cero. 🎉

Cierre — El Ciclo de Vida

#	Fase	Qué hicimos hoy
1	Requisitos	Definimos la API (4 endpoints)
2	Diseño	Arquitectura client-server
3	Implementación	9 archivos creados desde cero
4	Testing	Swagger, curl, Postman, frontend
5	Deployment	(Futuro) Docker + hosting
6	Mantenimiento	(Futuro) Features + fixes

El software no es solo código: es requisitos → diseño → código → testing → deploy → mantenimiento.

Ejercicios para Llevar a Casa

● Comprensión

1. ¿Cuántos endpoints tiene la API?
2. Modificá el health check.
3. Eliminá `created_at` del modelo.

● Aplicación

4. Agregá campo `priority`.
5. Agregá endpoint de estadísticas.
6. Creá filtros en el frontend.

● Análisis



Construiste tu primera app fullstack **desde cero** en 90 minutos.

Backend	Frontend	Swagger
<code>uv run main.py</code>	<code>pnpm dev</code>	<code>localhost:8000/docs</code>

La Universidad te da el mapa. El recorrido lo hacés vos.